

From Prompt Engineering to Loop Engineering: The Evolution of the Agentic Stack

Blog Outline — Long-Form Technical Post

Working Title

From Words to Systems: How the Agentic Stack Evolved from Prompt Engineering to Loop Engineering

Alternative: "Stop Prompting, Start Choreographing: The Four-Layer Evolution of AI Agent Architecture"

Opening Hook

"You shouldn't be prompting coding agents anymore. You should be designing loops that prompt your agents."

— Peter Steinberger, June 2026 ([MarkTechPost](#))

"My job is to write loops."

— Boris Cherny, Head of Claude Code, Anthropic ([Tweeny.ai](#))

Frame the thesis in the opening paragraph: The most important code in an agent system is not the agent itself — it is the code that wraps, constrains, and directs it. Over four years, the unit of engineering has shifted from a single prompt, to a context window, to a runtime harness, to an autonomous loop. Each layer didn't replace the previous; it absorbed it. This post maps the full stack, explains why each layer emerged, and argues that harness engineering is the stage and loop engineering is the choreography of modern autonomous systems.

Section 1 — The Thesis: Words Cannot Constrain Compute

Core Argument

LLMs are probabilistic reasoning engines. Production systems demand deterministic behavior. This fundamental mismatch is what every layer of the agentic stack attempts to resolve ([YouTube — Soochow University Workshop](#)).

- **Prompt engineering** assumed you could phrase your way to reliability. You cannot.
- **Context engineering** recognized that the problem wasn't wording — it was what the model could see.

- **Harness engineering** recognized that the problem wasn't visibility — it was what the model could touch.
- **Loop engineering** recognized that the problem wasn't a single action — it was the entire cycle of acting, evaluating, and continuing.

Key Framing for the Reader

Each layer is a different **unit of control**:

- A prompt controls one model response.
- Context controls one model invocation's information environment.
- A harness controls one agent's execution boundary.
- A loop controls one agent's behavior cycle over time.

Each layer preserves the layer beneath it. A prompt doesn't disappear once a loop is built around it — it stops being the thing typed by hand ([MarkTechPost](#)).

Section 2 — The Four Layers: Definition and Emergence

Layer 1 — Prompt Engineering (2022–2023)

What it is: Writing and structuring the instruction for a single model call. Anthropic's guidance is to separate system prompts into labeled sections — background, instructions, tool guidance, output description — delineated with XML tags or Markdown headers. Supply the minimal set of information that fully specifies expected behavior ([MarkTechPost](#)).

Why it emerged: Early LLMs (GPT-3, ChatGPT) were single-turn. The only lever was language.

What it solved: Getting a model to produce a useful single response.

The wall it hit: "Linguistic optimism" — the assumption that the model will behave if you phrase instructions clearly enough. In multi-hour autonomous execution, phrasing cannot prevent hallucination, unauthorized actions, or context drift ([YouTube — Harness Engineering](#)).

Layer 2 — Context Engineering (2024–2025)

What it is: Designing exactly what the agent sees — not just the prompt, but the entire token configuration in the context window. Includes RAG pipelines, file attachments, context restructuring into clean JSON payloads, prompt caching to save up to 90% in token costs ([YouTube — Soochow Workshop](#)).

Why it emerged: As agents moved from single-turn to multi-step, the context window became a finite resource. The engineering problem shifted from "finding the right words" to "optimizing the utility of tokens against model constraints" ([MarkTechPost](#)).

What it solved: Keeping the agent oriented across step 5, step 15, step 30.

The wall it hit: Context rot — the gradual degradation of reasoning quality as context fills with accumulated tool outputs, prior responses, and stale instructions ([AgenticPrep](#)).

Layer 3 — Harness Engineering (2025–2026)

What it is: The discipline of designing the scaffolding — context delivery, tool interfaces, planning artifacts, verification loops, memory systems, and sandboxes — that surrounds an AI agent and determines whether it succeeds or fails on real tasks ([GitHub — awesome-harness-engineering](#)).

The core equation: **Agent = Model + Harness** ([YouTube — Harness Engineering Explained](#)).

A harness is the deterministic wrapper around a probabilistic model. It reduces the space in which the model can be wrong ([YouTube — The Missing Execution Layer](#)).

Why it emerged: Production agents needed to run for hours, touch real systems, and make irreversible decisions. The model couldn't self-regulate. As Martin Fowler's team and the AI Engineer World's Fair formalized: the most important code in an agent system is the code that wraps, constrains, and directs the agent ([product.engineer](#)).

What it solved: Turning a model that can be talked past instructions into an agent that physically cannot execute outside its boundary.

The wall it hit: A harness makes a single agent reliable, but doesn't answer: when does it act? when does it stop? who escalates? That's the loop's job.

Convergence signal: OpenAI, Anthropic, LangChain, and Martin Fowler's team each published findings converging on the same conclusion: environment quality determines agent output quality more than model capability or prompt sophistication ([AgentPatterns.ai](#)).

Layer 4 — Loop Engineering (2026–present)

What it is: The software engineering discipline of designing the structural controls around an agentic loop: state machines, iteration limits, tool boundaries, verification gates, and human escalation paths. It makes loops that are not only capable but also safe, auditable, and deterministic enough for production ([Tweeny.ai](#)).

Why it emerged: Peter Steinberger's June 2026 post — "You shouldn't be prompting coding agents anymore. You should be designing loops that prompt your agents" — crystallized what frontier practitioners had already been doing. Boris Cherny confirmed: "My job is to write loops." Addy Osmani named the practice: "Loop engineering is replacing you as the person who prompts the agent" ([Tweeny.ai](#)).

What it solves: Moving from a human who prompts the agent at each step to a system that runs on timers, triages issues, writes code, runs tests, and files PRs without a human in the middle ([YouTube — What is Loop Engineering](#)).

Core pattern: The Maker/Checker — the agent that writes code should never be the one grading it. Sub-agents make asynchronous execution trustworthy ([YouTube — What is Loop Engineering](#)).

Section 3 — Comparative Matrix: The Four Layers Side by Side

Layer	Objective	Primary Control Point	What It Enables	Limitation / Failure Mode	Best-Fit Use Case
Prompt Engineering	Elicit a useful single response from the model	The instruction text — system prompt, user query, output format	Single-turn question answering, content generation, classification	"Linguistic optimism" — assumes clear phrasing guarantees compliance; fails under multi-step autonomy, context drift, and adversarial inputs	Chatbots, content generation, simple Q&A, prototyping
Context Engineering	Optimize what the agent sees at each invocation	The token budget — RAG pipelines, file attachments, context compaction, prompt caching	Multi-step agents that survive step 30, retrieval-augmented workflows	Context rot — accumulated tool outputs and stale instructions degrade reasoning quality over long sessions; finite window limits	RAG systems, coding assistants, document analysis, multi-turn conversations
Harness Engineering	Build the deterministic runtime boundary around the model	The execution environment — tools, permissions, sandboxes, memory, validation gates, CI guardrails	Production-grade agents that can touch real systems safely for hours	Makes a single agent reliable but doesn't define when to act, when to stop, or how to escalate; harness design can overfit to specific model versions	Production coding agents (Claude Code, Codex), autonomous workflows with real system access
Loop Engineering	Design the autonomous behavior cycle — observe, decide, act, evaluate, recover, continue/stop	The loop contract — cadence, triggers, delegation, retry policies, escalation paths, verification gates	Fully autonomous systems that run on timers, triage issues, write code, run tests, and file PRs without human initiation	Complexity of orchestration; debugging distributed multi-loop systems; ensuring safety when no human is in the loop	Background agents, CI/CD automation, autonomous issue triage, self-improving systems

Section 4 — Harness Engineering: The Essential Stage

Key Talking Points

- 1. The stage metaphor.** A model is a performer. The harness is the stage — the props, the exits, the safety nets, the lighting, the stage manager. Without the stage, the performer improvises in an empty room and eventually walks off the edge. The harness is every piece of code, configuration, and execution logic that isn't the model itself ([YouTube — Harness Engineering For AI Agents](#)).
- 2. Words cannot constrain compute.** This is the foundational insight. A prompt is a suggestion. A harness is a physical boundary. The agent cannot talk its way past a harness the way it can talk its way past a prompt instruction. The harness operates at the system level, outside the model's reasoning process, where it cannot be reasoned around, ignored, or misinterpreted ([product.engineer](#)).
- 3. The seven layers of a production harness.** A complete harness includes: Normalization, Context Orchestration, the Constraint Layer, Gated Execution, Validation & Repair, Graceful Degradation, and State Management. The PEV (Plan-Execute-Verify) loop separates planning from execution and uses structured feedback for autonomous self-correction ([YouTube — The Missing Execution Layer](#)).
- 4. The five structural primitives.** LangChain's breakdown: filesystem (durable state + collaboration surface), code execution (autonomous problem-solving), sandbox (isolation + verification), memory (cross-session persistence), and context management (compaction against context rot) ([GitHub — awesome-harness-engineering](#)).
- 5. Classification is the harness's job, not the model's.** Before an agent executes any action, the harness classifies it. If you ask the agent to self-classify its actions as "safe" or "dangerous," you're asking the same system that wants to take the action to evaluate whether it should. That's a conflict of interest built into the architecture ([product.engineer](#)).
- 6. The calibration framework.** Start restrictive — new agents begin with read-only permissions and mandatory checkpoints on every write. Measure violation rate — track how often the agent attempts actions outside its boundary. Expand where violations are safe — if the agent repeatedly tries something and that action would have been fine, expand the permission ([product.engineer](#)).
- 7. Evidence of convergence.** LangChain improved Terminal Bench 2.0 from 52.8% to 66.5% through pure harness changes — no model change. OpenAI shipped roughly one million lines of production code without manually written source in a five-month experiment. The enabler was environment design ([AgentPatterns.ai](#)).
- 8. The cross-cutting mental model.** Many engineers treat context, harness, and loop as a sequential pipeline. The correct model is cross-cutting, not strictly concentric: the **harness** is the runtime/stage boundary — what the agent can see, touch, execute, persist, and recover from. The **loop** is the temporal/choreography control — when the agent runs, how it iterates, verifies, retries, escalates, and stops. A loop should invoke or operate through a harness; it

should never bypass it. Together they form two orthogonal axes: the harness constrains *what*, the loop governs *when* ([YouTube — Soochow Workshop](#)).

9. The UN governance angle. Agentic capability is best understood at the level of the model-harness pair. A strong model can perform poorly in long-horizon tasks if it lacks stable interfaces, structured memory, scoped permissions, and recovery mechanisms. A more modest model can perform more dependably when the surrounding harness reduces ambiguity, externalizes memory, and constrains destructive actions ([United Nations University](#)).

10. Why it's the essential stage, not the final destination. The harness makes an agent safe. It doesn't make an agent autonomous. For that, you need the loop — the choreography that decides when the agent enters the stage, what it does between acts, and when the curtain falls.

Section 5 — Loop Engineering: The Choreography for Autonomous Systems

Key Talking Points

1. The choreography metaphor. If the harness is the stage, the loop is the choreography — the sequence of movements, the timing, the entrances and exits, the coordination between performers. A stage without choreography is a safe room where nothing happens. Choreography without a stage is a dance that can't be performed safely. Both are necessary; neither is sufficient alone.

2. The agentic loop defined. A repeating, goal-directed cycle in which an AI agent takes an action, receives feedback from its environment, reasons about that feedback, and decides its next move, continuing autonomously until a verifiable completion condition is satisfied. Not a chain (A to B to C). Dynamic — the agent observes, adapts, and retries without human intervention at each step ([Tweeny.ai](#)).

3. The three functional stages of a well-engineered loop. Every solid loop has: (1) Observation — the agent perceives its environment and current state. (2) Decision — the agent reasons about what to do next. (3) Action and Evaluation — the agent acts, then a separate process evaluates the result against a completion condition. The loop continues or terminates based on that evaluation ([Tweeny.ai](#)).

4. The Loop Contract. The loop is governed by a formal specification — an eleven-field contract that defines the boundaries and goals of an autonomous loop: triggers, cadence, iteration limits, tool boundaries, verification gates, delegation rules, retry policies, escalation paths, state persistence, acceptance criteria, and termination conditions ([DeepWiki — awesome-loop-engineering](#)).

5. The Maker/Checker pattern. The agent that writes code should never be the one grading it. A separate, independent process (or agent) verifies the output. This separation of concerns is what makes asynchronous execution trustworthy — you're not relying on the same probabilistic system that produced the work to also judge it ([YouTube — What is Loop Engineering](#)).

6. The six core pillars. Every solid loop is built from: Automations (event-driven triggers), Worktrees (isolated execution environments), Skills (reusable capability modules),

Plugins/MCP Connectors (external tool interfaces), Sub-Agents (delegated parallel workers), and External Memory (like [AGENTS.md](#) for persistent context) ([YouTube — What is Loop Engineering](#)).

7. From human-in-the-loop to human-on-the-loop. The harness engineer designs and maintains agent environments rather than inspecting individual outputs. This is the "humans on the loop" framing — humans govern the loop's structure, not its individual iterations ([GitHub — awesome-harness-engineering](#)).

8. The three-era timeline. Era 1 (2022): "What do I type?" — the unit of work is a prompt. Era 2 (2024–2025): "What surrounds the prompt?" — the unit of work is a context. Era 3 (2026): "What system do I build?" — the unit of work is a loop ([YouTube — What is Loop Engineering](#)).

9. Self-improving harnesses as the frontier. The Self-Harness framework (Zhang et al. 2026) uses a propose-evaluate-accept loop where AI agents iteratively improve their own scaffolding by mining execution traces. The Meta-Harness (Lee et al. 2026) goes deeper: the optimized object is the code that determines what information should be stored, retrieved, and presented. A harness for optimizing harnesses ([Lilian Weng — Lil'Log](#)).

10. Why loop engineering is the choreography, not the conductor. The loop doesn't dictate what the agent thinks — that's the model's job. It doesn't dictate what the agent can touch — that's the harness's job. The loop dictates the rhythm: when to act, when to pause, when to verify, when to escalate, when to stop. It's the temporal architecture of autonomy.

Section 6 — Timeline Graphic Concept: From Manual Prompting to Automated Triggering

Visual Description for a Designer or Diagramming Tool

Title: "The Shifting Initiation Point: 2022–2026"

Format: Horizontal timeline (left to right), with a gradient bar along the bottom axis labeled "Human Initiation → System Initiation." The gradient shifts from warm (orange/red, left side = human) to cool (blue/teal, right side = system).

Timeline Stops (left to right):

Stop	Year	Milestone	Who Initiates	Visual Element
1	2022	Manual Prompting — user types a prompt, model responds, user reads	Human, every turn	A speech bubble icon with a cursor
2	2023	Prompt Templates — structured prompts with system/user separation, few-shot examples	Human, with tooling	A template/form icon
3	2024	Tool-Augmented Workflows — model calls functions, but human triggers each session	Human initiates, model acts within turn	A wrench + arrow icon

Stop	Year	Milestone	Who Initiates	Visual Element
4	2025	Context-Managed Agents — RAG, memory, multi-step reasoning with managed context windows	Human initiates, system sustains	A layered window icon
5	2025–26	Harnessed Agents — agent runs in a sandbox with scoped permissions, validation gates, and state management	Human initiates, harness governs	A shield/container icon
6	2026	Event-Triggered Loops — agent activates on webhooks, timers, CI events, or issue creation	System initiates, human reviews	A gear + lightning bolt icon
7	2026+	Autonomous Multi-Loop Systems — nested loops with sub-agents, self-improvement, and human-on-the-loop governance	System initiates, system governs, human oversees architecture	A network of interconnected loops icon

Axis Annotation:

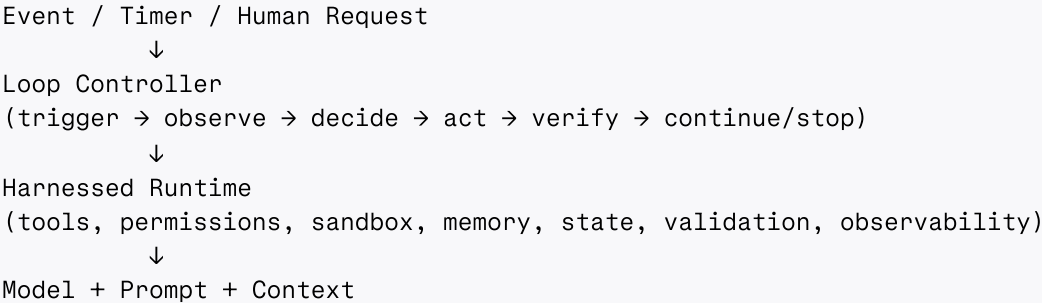
- **X-axis (bottom):** Time — 2022 to 2026+
- **Y-axis (left, implicit):** Autonomy Level — increasing upward
- **Bottom gradient bar:** Labeled "Initiation Point" with arrow pointing right, annotated "Manual Prompting" on the left end and "Automated Triggering" on the right end

Overlay Callouts (floating annotations above the timeline):

- Above Stop 1–2: "The era of linguistic optimism — phrasing was the only lever"
- Above Stop 3–4: "The context era — what the agent sees becomes the engineering problem"
- Above Stop 5: "The harness era — physical boundaries replace linguistic suggestions" (cite [product.engineer](#))
- Above Stop 6–7: "The loop era — systems prompt the agent, not humans" (cite [Tweeny.ai](#))

Secondary Visualization (inset or below the timeline):

A cross-cutting architecture diagram showing the two orthogonal axes — the harness (runtime boundary) and the loop (temporal control):



The loop invokes the harness at each iteration. The harness constrains what the model can do inside each iteration. Neither bypasses the other. The loop governs *when*; the harness

constrains *what*. Together they form the complete execution environment for an autonomous agent.

Section 7 — The Cross-Cutting Mental Model: Why Pipeline Thinking Fails

Key Argument

Many engineers treat context, harness, and loop as a sequential pipeline — data flows from one stage to the next. This is wrong. The correct model is cross-cutting: the harness and the loop are orthogonal axes that operate simultaneously, not layers in a stack. The harness constrains what the agent can do at any moment; the loop governs when and how the agent acts over time ([YouTube — Soochow Workshop](#)).

Why This Matters

- **Pipeline thinking** implies you can bypass a layer. If data flows A to B to C, you can skip B.
- **Cross-cutting thinking** implies you cannot. The harness is always present as the execution boundary. The loop is always present as the control flow. Every harness has at least implicit control flow, but autonomy begins when the loop becomes explicit — when a human is no longer the one triggering each iteration.

Analogy

Think of it like an operating system stack:

- The **prompt** is a single syscall.
- The **context** is the process's memory space.
- The **harness** is the kernel's permission system — what the process can touch.
- The **loop** is the scheduler — when the process runs, for how long, and when it gets preempted.

These aren't stages in a pipeline. They're orthogonal, always-on systems that interact at every step. The harness constrains *what* the agent can do; the loop governs *when* and *how* it does it.

Section 8 — Practical Implications for Teams Building Agent Systems

Talking Points

1. **Evaluate the model-harness pair, not the model alone.** A strong model can perform poorly in long-horizon tasks if it lacks stable interfaces, structured memory, scoped permissions, and recovery mechanisms. The relevant object of evaluation is the system formed by the model, its tools, its memory, its permissions, its orchestration loop, and its recovery procedures ([United Nations University](#)).

2. **Document the harness as a first-class artifact.** Institutions should document the harness as a first-class runtime configuration, adopt a baseline for reliable long-running execution, make state-changing actions interruptible, resumable, and reversible, and apply least-privilege tool governance with explicit approval gates ([United Nations University](#)).

3. **The repository is the primary interface.** If something is not in the repo, it does not exist for the agent. Repository legibility — how easily an agent can find, read, and act on project knowledge — determines the capability ceiling. [AGENTS.md](#) as a compact index (~100 lines) pointing to deeper resources, not a monolithic knowledge dump ([AgentPatterns.ai](#)).

4. **Start with one model, a few narrow tools, and an explicit acceptance check.** Add retries after traces show transient call failures. Add a progress handoff when resumed sessions repeat work. When you test either change, keep the model version, tasks, grader, and total budget fixed. Remove the component when it does not improve the measured result ([Slava Dubrov](#)).

5. **The co-evolution warning.** Models trained with specific harnesses can become overfitted to those designs. Harness architecture choices have lasting consequences beyond the immediate task ([GitHub — awesome-harness-engineering](#)).

Section 9 — What Comes Next: Graph Engineering and Beyond

Brief Forward Look

The stack may not end at loops. MarkTechPost identifies a potential fifth layer — **graph engineering** — which controls the organization of many agents rather than one agent's behavior cycle. Each layer preserves the layer beneath it, and the pattern suggests the next unit of control is multi-agent topology ([MarkTechPost](#)).

Closing Thought

The evolution from prompt engineering to loop engineering is not a story of replacement. It's a story of containment. Each layer emerged because the previous one could not contain the complexity of real-world execution. The prompt contains the instruction. The context contains the information. The harness contains the execution. The loop contains the behavior over time. And the question for the next era is: what contains the loops?

Sources

- [MarkTechPost — Prompt Engineering vs Loop Engineering vs Graph Engineering](#)
- [Tweeny.ai — From Manual Prompting to Agentic Loops](#)
- [product.engineer — Harness Engineering: When Humans Steer and Agents Execute](#)
- [GitHub — awesome-harness-engineering](#)
- [AgentPatterns.ai — Harness Engineering for Building Reliable AI Agents](#)
- [Lilian Weng.\(Lil'Log\) — Harness Engineering for Self-Improvement](#)

- [United Nations University — Engineering and Governing the Agent Harness](#)
- [Slava Dubrov — Harness engineering for AI agents: designing the loop around the model](#)
- [DeepWiki — awesome-loop-engineering](#)
- [AgenticPrep — Building Agentic Systems Blog](#)
- [YouTube — Beyond Prompt Engineering: Designing the Enterprise Agentic Stack](#)
- [YouTube — Harness Engineering Explained](#)
- [YouTube — Harness Engineering: The Missing Execution Layer](#)
- [YouTube — What is Loop Engineering?](#)
- [YouTube — Harness Engineering For AI Agents](#)
- [Microsoft Learn — Agent Harnesses](#)